The `numpy` module

Introduction

Numpy is a Python package, it stands for numerical python. Using `numpy` we can perform operations such as:

- Mathematical and logical operations on arrays
- Operations related to linear algebra
- Built-in functions for random number generation

In [1]:
```
import numpy as np
```

The `ndarray` object

The most important object defined in NumPy is an N-dimensional array type called `ndarray`:

- It describes the collection of items of the same type
- Items in the collection can be accessed using a zero-based index

In [2]:
```
L = [1, 2, 3]
print(L)
a = np.array(L)
print(a)
```

```
[1, 2, 3]
[1 2 3]
```

In [3]:
```
L = [[1, 2, 3], [4, 5, 6]]
a = np.array(L)
print(a)
```

```
[[1 2 3]
 [4 5 6]]
```

Computing the length of the array in `numpy` is done via the `shape` attribute:

In [4]:
```
print(a.shape)
```

```
(2, 3)
```

Unlike Python lists, you can specify the data-type of the items in a `numpy` array. Python automatically casts all the items to the specified `dtype`. Once you specify the `dtype`, mixed types will not be allowed.

In [5]:
```
a_mixed = np.array([1, 'CSC243'])
print(a_mixed)
a_float = np.array([1, 2, 3 ], dtype=float)
print(a_float)
a_float = np.array([1.5, 2.7, 3.1 ], dtype=int)
print(a_float)
```

```
['1' 'CSC243']
[1. 2. 3.]
[1 2 3]
```

Creating `ndarray` objects

Numpy `ndarrays` are created:

- As empty : empty function allocates memory without clearing it. The values inside will be arbitrary "**garbage data**" left over in your computer's RAM. You must overwrite them before reading from the array.
- As zeros
- As ones
- From existing data

In [6]:
```
a = np.empty((3,4), dtype=float)
print(a)
a = np.zeros((2,4))
print(a)
a = np.ones((3,2))
print(a)
a = np.ones((3,), dtype=int)
print(a)
```

```
[[1.13025232e-311 3.16202013e-322 0.00000000e+000 0.00000000e+000]
 [1.11260619e-306 6.38707346e-067 2.38485151e+180 2.72577673e-056]
 [5.69654228e-038 6.70750965e+170 8.93338217e+165 4.91321898e-062]]
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]]
[[1. 1.]
 [1. 1.]
 [1. 1.]]
[1 1 1]
```

Ranges

You can create ranges of data through:

- `np.arange(start, stop, step, dtype)` : returns an `ndarray` object containing evenly spaced values within a given range.
- `np.arange(stop, dtype)` : returns an `ndarray` object containing evenly spaced values from 0 till stop-1.

- `np.linspace(start, stop, num, dtype)` : instead of step size, the number **num** of evenly spaced values in the interval is specified.

In [7]:
```
r = np.arange(1.5, 2, 0.1)
print(r)
r = np.arange(5)
print(r)
r = np.linspace(10, 50, 5)
print(r)
```

```
[1.5 1.6 1.7 1.8 1.9]
[0 1 2 3 4]
[10. 20. 30. 40. 50.]
```

One advantage of using `numpy.arange` instead of range is that the step-size can be of type `float` in this case.

Math functions using `numpy`

Mathematical functions can also be accessed from the `numpy` module (sin, cos, exp, ceil, floor, etc.). The advantage of using `numpy` functions is that we can pass a full array as an input (in contrast to the functions from the `math` module, which take a single value as input).

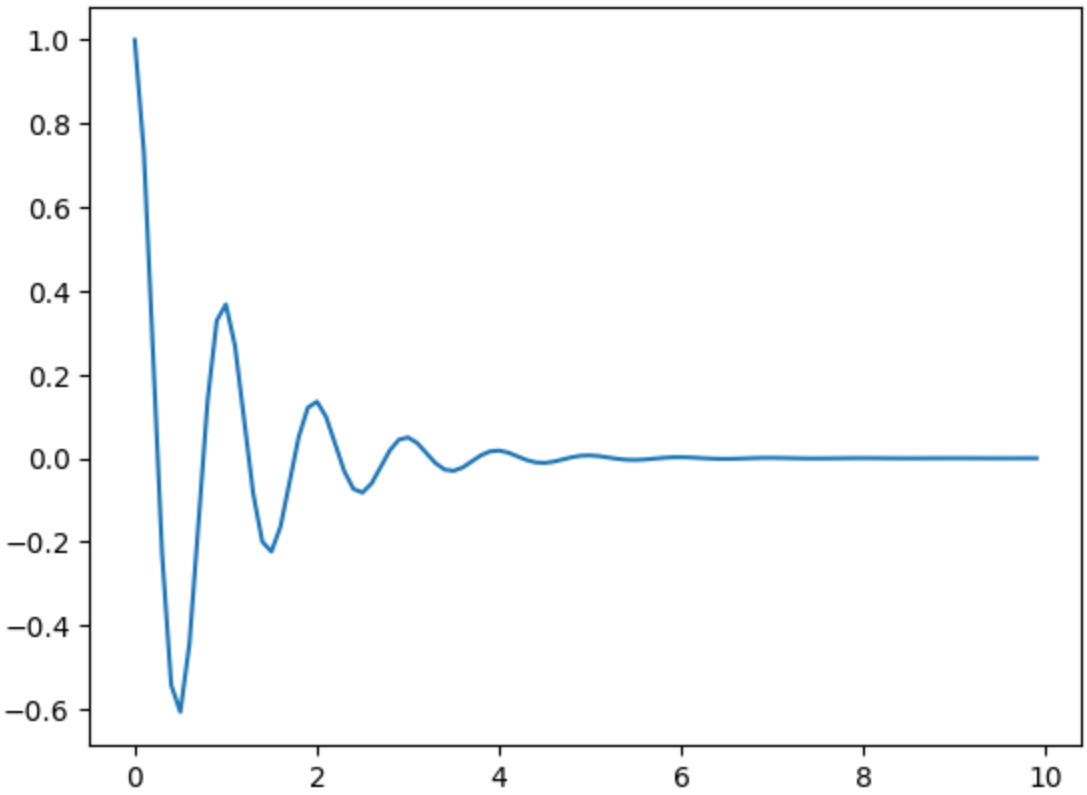
Example: plotting exp(-x)*cos(2πx), for x = [0, 0.1, 0.2, ..., 9.9]

In [8]:
```
from math import pi
import matplotlib.pyplot as plt

X = np.arange(0, 10, 0.1)
Y = np.exp(-X)*np.cos(2*pi*X)
print(Y)
plt.plot(X, Y)
```

```
[ 1.00000000e+00  7.32028848e-01  2.53001717e-01 -2.28925420e-01
 -5.42300309e-01 -6.06530660e-01 -4.43997940e-01 -1.53453298e-01
  1.38850286e-01  3.28921764e-01  3.67879441e-01  2.69298364e-01
  9.30741301e-02 -8.42169556e-02 -1.99501135e-01 -2.23130160e-01
 -1.63337714e-01 -5.64523135e-02  5.10801656e-02  1.21003555e-01
  1.35335283e-01  9.90693315e-02  3.42400590e-02 -3.09816865e-02
 -7.33923659e-02 -8.20849986e-02 -6.00885870e-02 -2.07676456e-02
  1.87913428e-02  4.45147201e-02  4.97870684e-02  3.64455703e-02
  1.25962138e-02 -1.13975255e-02 -2.69995426e-02 -3.01973834e-02
 -2.21053558e-02 -7.63998984e-03  6.91294868e-03  1.63760504e-02
  1.83156389e-02  1.34075760e-02  4.63388808e-03 -4.19291532e-03
 -9.93257663e-03 -1.11089965e-02 -8.13210594e-03 -2.81059519e-03
  2.54313170e-03  6.02441225e-03  6.73794700e-03  4.93237158e-03
  1.70471216e-03 -1.54248735e-03 -3.65399074e-03 -4.08677144e-03
 -2.99163459e-03 -1.03396019e-03  9.35565868e-04  2.21625741e-03
  2.47875218e-03  1.81451810e-03  6.27128556e-04 -5.67449383e-04
 -1.34422807e-03 -1.50343919e-03 -1.10056086e-03 -3.80372697e-04
  3.44175449e-04  8.15315539e-04  9.11881966e-04  6.67523905e-04
  2.30707703e-04 -2.08752962e-04 -4.94513872e-04 -5.53084370e-04
 -4.04873715e-04 -1.39931295e-04  1.26615072e-04  2.99937825e-04
  3.35462628e-04  2.45568321e-04  8.48726207e-05 -7.67959230e-05
 -1.81921487e-04 -2.03468369e-04 -1.48944716e-04 -5.14778466e-05
  4.65790818e-05  1.10340959e-04  1.23409804e-04  9.03395368e-05
  3.12228923e-05 -2.82516412e-05 -6.69251749e-05 -7.48518299e-05
 -5.47936988e-05 -1.89376414e-05  1.71354866e-05  4.05921705e-05]
```

Out[8]: [<matplotlib.lines.Line2D at 0x214b3bb03a0>]



Statistical Operations

We can compute the minimum and maximum through `np.amin(a, axis=)` and `np.amax(a, axis=)`. The `axis` keyword specifies the dimension over which min/max is computed. If not specified, the min/max will be computed over the whole array.

In [9]:
```
a = np.array([[7,2], [3,4], [5,6]])
print("a =\n" + str(a))
print("min:", np.amin(a))
print("minima (axis=0):", np.amin(a, axis=0))
print("minima (axis=1):", np.amin(a, axis=1))
```

```
a =
[[7 2]
 [3 4]
 [5 6]]
min: 2
minima (axis=0): [3 2]
minima (axis=1): [2 3 5]
```

We can also compute the average (mean), median, standard deviation and variance. We use in these cases `np.mean()`, `np.median()`, `np.std()`, and `np.var()`, respectively. All these functions also admit an `axis` keyword parameter like `np.amin`.

Generating Random Numbers

We can generate random numbers using `numpy.random`. Here are a few examples:

In [10]:
```
import numpy.random as rand

x = rand.uniform(1, 10)
print(x)
```

```
6.694346774610491
```

The `random.uniform(x, y)` function generates a uniformly-distributed random `float` in the real interval [x, y]. The default values are `x=0, y=1`.

The `random.randint` generates a random integer between `a` and `b` inclusive. Multiple runs of the same code give different random numbers (that is whole point).

In [11]:
```
a = np.random.randint(3,500)
print("a=",a)
```

```
a= 301
```

In [12]:
```
a = 1
b = 5
y = rand.randint(a, b+1)
print(y)
```

```
5
```

In [13]:
```
arrayD = np.random.randint(3,500,10)
print(arrayD.shape)
```

```
print ("arrayD=" , arrayD)
```

```
(10,)
arrayD= [206 408 419 453 257 392 414 457 346 254]
```

Pseudo-random values

Numpy can be used to generate pseudo-random values from various distributions.

In particular, a very useful distribution is the standard normal (zero mean and unit variance).

Let's generate two vectors sampled from the normal distribution, using a length parameter that we'll be able to change if needed.

The **numpy.random.randn()** function in Python's NumPy library is used to generate random numbers sampled from the standard normal distribution.

In [14]:
```
length = 50

vecA = np.random.randn(length)
vecB = np.random.randn(2*length)

print("vecA:" , vecA)
print("vecB:" , vecB)
```

```
vecA: [ 0.28706386 -0.72043549 -0.65119943  0.79195234  1.12468945  0.71685308
 -0.81576559 -0.99106233 -0.54136839 -0.50231872 -1.44407445 -0.38292891
 -0.56055403 -0.22995435  1.04406663  2.75191365  0.54808549 -0.73719433
  0.29491912  1.29069887 -1.16101149  0.44841853  1.56126396 -0.81376894
 -0.58266247 -0.42907048  0.20139775 -0.40072542 -0.53224356 -0.21341644
 -1.39641112 -1.70072299 -0.55151361  0.22505915  0.85720103 -0.07165137
  0.30014491  0.76154698 -0.66473259  0.78654389 -0.58652328 -1.06017239
  1.36957269  0.33176451 -0.9010677   1.52963242  0.75397125  0.04970914
 -1.04232536 -0.28825472]
vecB: [-1.47383643e+00 -1.06224027e+00  1.05317781e+00 -9.60567067e-01
 -9.34296790e-01 -8.50906013e-01  5.33830528e-01 -6.57955399e-01
  1.70528795e+00 -4.22347958e-01  6.97455549e-02 -3.78811179e-01
 -4.63912697e-01  5.67312298e-01 -7.24247561e-01  4.75408176e-01
 -6.79847545e-01 -6.67576262e-02  5.04193071e-01  2.02995710e-01
 -7.07983363e-01 -4.33361074e-01 -2.20867664e+00  3.03807247e-01
  5.12084249e-01  1.17277554e+00  1.26595864e+00  1.14521375e+00
  7.25602898e-01 -8.69357827e-01 -4.14923316e-01 -1.26728429e-01
  7.53098978e-01 -3.39719712e-01 -5.37855381e-01  1.33372817e+00
  1.92256926e+00 -5.16426979e-01  1.06285467e+00 -2.10602175e-02
  5.66651719e-01 -1.40319744e+00  2.72092541e+00  2.31252430e+00
  2.66617826e-01 -2.83780600e-01  1.25347189e-01 -3.57702931e-01
  1.37007740e+00 -8.62922000e-01 -1.60349579e+00 -4.04552430e-01
  1.79636398e-01 -5.41942351e-01 -7.93086845e-01 -1.54970813e+00
 -2.92628022e+00  2.33853524e-01  6.73969273e-02 -2.82202071e-01
 -7.23427292e-01  6.88127030e-01 -6.59132536e-01  1.42999993e+00
 -1.20807235e+00 -9.03145332e-01  1.02200202e+00  5.53540260e-01
  5.14737758e-01  2.55117464e-01 -1.01167094e+00 -8.27273226e-01
 -1.62052204e-02  1.72568917e+00  1.05526326e+00 -1.17464600e-01
  5.25606393e-01  5.59762332e-01 -5.58934851e-01  1.33163731e+00
  1.19871368e+00 -1.77886311e+00  1.70080013e+00 -2.00568280e+00
 -2.49808160e-03 -9.76995517e-01 -1.63482705e+00 -5.75864385e-01
  1.30749616e-01 -1.65162905e+00 -8.01922678e-02 -5.77017938e-01
 -1.71688030e+00 -1.45213595e+00 -8.84065561e-01  2.71527264e-01
  7.47737634e-01 -1.19355553e+00  1.65188024e+00  2.95544807e+00]
```

vecA and vecB are numpy *arrays*. One of their attributes can be fetched to check their *shape*

In [15]:
```
print(vecA.shape)
print(vecB.shape)
```

```
(50,)
(100,)
```

Numpy arrays can be vectors as well as matrices, or any tensor. For example the following code will create tensors with 3 dimensions using the standard normal

In [16]:
```
arrayC = np.random.randn(3,10,4)
print(arrayC.shape)
print(arrayC)
```

```
(3, 10, 4)
[[[-2.2962212  -0.37420983 -0.66432402 -1.04833931]
  [ 0.68778506  0.36756297  0.33696081  0.01840315]
  [ 0.24316051 -0.15032262 -0.02572335  0.09637169]
  [-0.36951088  0.5417221   1.03575959 -1.89102518]
  [-0.40768474  0.4363188  -0.295416    0.0331446 ]
  [ 0.34728723 -1.27594267 -0.55839009 -0.54592466]
  [-0.16672473  0.78499379 -2.15940051 -1.50087861]
  [-0.54233128 -0.94897068  0.12929427  0.0705635 ]
  [-0.83758202  0.91758317  0.5214451  -1.10031472]
  [-0.33363026 -0.80205849 -0.74720987  0.73765816]]

 [[ 0.82460451  0.13312381 -0.38060562 -0.31853689]
  [-1.52347416  1.44479612  0.26903899  1.64063369]
  [ 0.34801205 -0.68161593  0.32914115  0.85970584]
  [-0.3935214   1.23900938  0.6205688   0.03450764]
  [ 0.84607867  0.56139213 -0.29460018  1.79161983]
  [ 1.81649833 -1.19110829 -1.22046029  0.17240285]
  [ 0.77930059 -0.23930921 -0.68853379  0.51459319]
  [-0.35704417 -0.85865052 -0.10522822  0.28884014]
  [ 1.05105715  0.66659191 -0.4197388   0.74498645]
  [-0.3824333   2.25900592  0.68923745  0.54580407]]

 [[ 2.00004634 -0.10299633 -0.81940765  0.29102268]
  [-1.45166697 -1.8082227   0.37294128  0.20264008]
  [-0.14193326  0.78891386  0.3553395   0.27375932]
  [ 1.96289811 -0.42068723  1.57837375 -0.27449621]
  [ 0.57551811  0.76133291 -0.12883106  1.0707795 ]
  [-0.25847333 -0.52420756  0.81146779 -0.69500384]
  [ 0.50836823  0.16118334  1.07476716 -2.11518903]
  [ 0.96318683 -0.261241   -0.1944165   1.13957939]
  [ 0.26625662  0.52288141  0.45057335  0.98181666]
  [-1.27736297 -1.95707682  0.96609406 -0.37254734]]]
```

A very important features of arrays is the fact they can be used as *iterables*. For example, you can iterate over the dimensions of an array by simply "looping" over it using a *for* loop

In [17]:
```
for dim in arrayC:
    print(dim.shape)
```

```
(10, 4)
(10, 4)
(10, 4)
```

Also possible to enumerate along the dimension in order to get the index of the current "smaller" array

In [18]:
```
print('Initial shape is %d %d %d' % (arrayC.shape[0],arrayC.shape[1],arrayC.shape[2
print('Iterating over the first dimension using an index k')
for k,curdim in enumerate(arrayC):
    print('k = %d, shape is %d %d' % (k,curdim.shape[0],curdim.shape[1]))
```

```
Initial shape is 3 10 4
Iterating over the first dimension using an index k
k = 0, shape is 10 4
k = 1, shape is 10 4
k = 2, shape is 10 4
```

Another important operation when working with Numpy Arrays is *reshaping*. Essentially, *reshaping* consists in changing the organisation of the array (in terms of dimension), while keeping the same number of elements. For example, a 20x10 2D array can be converted into a 4x5x10 array

In [19]:
```
A = np.random.randint(1,50,(10,20))
print('Initial shape of A is %d x %d' % (A.shape[0],A.shape[1]))
print(A)
B = A.reshape((4,5,10))
print('B is A reshaped to %d x %d x %d' % (B.shape[0],B.shape[1],B.shape[2]))
print(B)
```

```
Initial shape of A is 10 x 20
[[15 46  2 43 11 45 31 44 48 41 41 44 47 39 47  8  2  5 34 33]
 [27 44  4 13 43  9 25 48 41 46 21 39 41 32 35  4 25 32 46 40]
 [23 22 36  5 11 30 40 28 33  4 17 38  6 24  4 31  9 15 38  5]
 [37 13 28 38 17  3 26 21 32 19 40 33 21 31 22 34 48 44 31  1]
 [ 2 28 19 22 48 38 26 22 10 39  3  7 27 10  7  6 26 17  7 45]
 [34 26 11 33 47 21 25 38 13  2 12 27 23 33 34 24 28 16 31 35]
 [49 41 43 46 23 29 25 42  1 38  3 19  2 21 35 34 14 11 32  3]
 [ 7 14 17 22 16 41 32 25  1 21 27 43  1 47 12  6 40 37 36 23]
 [13 32 49  5  3 24 12 29 42 45 37  4 41 35 23 40 11  9 38 14]
 [ 8 48 13 39 41 48 27 38  4 30 35 28 33  8 20  3 45 37 30 13]]
B is A reshaped to 4 x 5 x 10
[[[15 46  2 43 11 45 31 44 48 41]
  [41 44 47 39 47  8  2  5 34 33]
  [27 44  4 13 43  9 25 48 41 46]
  [21 39 41 32 35  4 25 32 46 40]
  [23 22 36  5 11 30 40 28 33  4]]

 [[17 38  6 24  4 31  9 15 38  5]
  [37 13 28 38 17  3 26 21 32 19]
  [40 33 21 31 22 34 48 44 31  1]
  [ 2 28 19 22 48 38 26 22 10 39]
  [ 3  7 27 10  7  6 26 17  7 45]]

 [[34 26 11 33 47 21 25 38 13  2]
  [12 27 23 33 34 24 28 16 31 35]
  [49 41 43 46 23 29 25 42  1 38]
  [ 3 19  2 21 35 34 14 11 32  3]
  [ 7 14 17 22 16 41 32 25  1 21]]

 [[27 43  1 47 12  6 40 37 36 23]
  [13 32 49  5  3 24 12 29 42 45]
  [37  4 41 35 23 40 11  9 38 14]
  [ 8 48 13 39 41 48 27 38  4 30]
  [35 28 33  8 20  3 45 37 30 13]]]
```

One property of numpy that is really important is broadcasting. The goal of broadcasting is to simplify the vectorization of certain operations when the vectors do not have the same shape. For example you can easily perform element-wise multiplication.

In [20]:
```
x = np.array([2,3])
y = np.array([[4,1],[9,10],[12,13]])
result = x*y
print("X: ",x)
print("Y: ")
print(y)
print("X shape is: ",x.shape)
print("Y shape is: ",y.shape)
print("Element-wise multiplication shape:", result.shape)
print("Element-wise multiplication result:")
print(result)
```

```
X:  [2 3]
Y:
[[ 4  1]
 [ 9 10]
 [12 13]]
X shape is:  (2,)
Y shape is:  (3, 2)
Element-wise multiplication shape: (3, 2)
Element-wise multiplication result:
[[ 8  3]
 [18 30]
 [24 39]]
```

Another very powerful tool in numpy is indexing. You can use either an integer vector or a boolean vector to choose which indexes you want to extract from your numpy tensor.

Consider that we want to extract all elements from the first line of your vector y that have a higher value than 1, you would have to do:

In [21]:
```
first_row = y[0]
print(first_row)
first_row_higher_than_one = first_row > 1
print("Result: ", first_row[first_row_higher_than_one])
```

```
[4 1]
Result:  [4]

Result:  [4]
```

You can also choose specific lines to query, for example if you want to query lines 0 and 2

In [22]:
```
rows = [0,2]
rows_result = y[rows]
values_higher_than_one = rows_result > 1
print("Result: ", rows_result[values_higher_than_one])
```

```
Result:  [ 4 12 13]
```

You can also save and load your numpy tensors using np.savez and np.load. This will be really important in the next courses as this enable you to generate your data only once instead of having to do all the calculations every time you need your data.

In [23]:
```
filename = "y.npz"
source_tensor = y
np.savez(filename,data=source_tensor)
```

In [24]:
```
loaded_npz = np.load(filename)
loaded_tensor = loaded_npz["data"]
print("Your tensor was loaded and contains: ", loaded_tensor)
```

```
Your tensor was loaded and contains:  [[ 4  1]
 [ 9 10]
 [12 13]]
 [[ 4  1]
 [ 9 10]
 [12 13]]
```

In [ ]: